

Reviewer Pack — Minimal Rank of Integer-Lattice Homology and Circuit Monoton...

Entry c266c5a3a6c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 08:12:45 UTC

Minimal Rank of Integer-Lattice Homology and Circuit Monotone Width Inequality

Entry ID: c266c5a3a6c4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 08:12:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Integer Lattice Homology) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For any given d -regular graph G , the minimal rank of the integer-lattice homology group $H(G; \mathbb{Z})$ is linearly correlated with its circuit monotone width $w_m(G)$, such that $|H(G; \mathbb{Z})| = \Theta(w_m(G))$.

Rationale (proposer's reasoning):

Integer-lattice homology provides a topological invariant capturing the connectivity of the graph at various resolutions, which may reveal subtle properties related to circuit complexity. The conjecture posits that the minimal rank of the homology group is directly linked to the circuit monotone width, suggesting a deep connection between topological and computational structures.

Taxonomy category: topological-invariants (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a35853b214e660e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all d -regular graphs G with $n \leq 40$ vertices and 30 independent seeds, the correlation coefficient between $|H(G; \mathbb{Z})|$ and $w_m(G)$ exceeds 0.7 across all seeds, and no seed produces a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:integer lattice homology AND circuit monotone width - arxiv:math.AT OR arxiv:cs.CC AND integer-lattice homology AND circuit monotone width inequality - title:linear correlation AND minimal rank AND integer-lattice homology AND circuit monotone width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            raise ValueError("Graph size must be a multiple of the degree")
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u, v = random.sample(range(n), 2)
                if (u, v) not in edges_added and (v, u) not in edges_added:
                    graph[u].append(v)
                    graph[v].append(u)
                    edges_added.add((u, v))
                    break
            return graph

    def circuit_monotone_width(graph):
        n = len(graph)
        max_width = 0
        for subset_size in range(1, n // 2 + 1):
            subsets = combinations(range(n), subset_size)
            for subset in subsets:
                subgraph = {i: [j for j in graph[i] if j in subset] for i in subset}
                width = len(subset) - len(set.union(*[set(graph[i]) for i in subgraph]))
                max_width = max(max_width, width)
            return max_width
```

```

def integer_lattice_homology(graph):
    n = len(graph)
    homology = [0] * (n + 1)
    homology[0] = 1
    for node in graph:
        neighbors = set(graph[node])
        new_homology = [0] * (n + 1)
        for i in range(n, -1, -1):
            if homology[i] > 0:
                new_homology[i - len(neighbors)] += homology[i]
                new_homology[i] -= homology[i]
        homology = new_homology
    return sum(homology)

n_values = [5, 10, 15, 20, 30, 40]
correlation_sum = 0
instances_tested = 0

for n in n_values:
    for _ in range(5):
        graph = generate_d_regular_graph(n, 3)
        homology_rank = integer_lattice_homology(graph)
        width = circuit_monotone_width(graph)
        if homology_rank == 0 or width == 0:
            continue
        correlation_sum += homology_rank * width
        instances_tested += 1

if instances_tested < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mean = correlation_sum / instances_tested
return {
    "metric_name": "correlation_coefficient",
    "metric_value": mean,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": False,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)

```

```

    print(f"TRIAL: {result}")

mean = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean} std=0 support_fraction=1")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std=0 support_fraction={support_fraction}")
else:
    counterexample = min((result["counterexample"] for result in results if result["conjecture_holds"] == False))
    first_failing_seed = next((result["seed"] for result in results if not result["conjecture_holds"]))
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2a93101.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2a93101.py", line 66, in run_trial
    graph = generate_d_regular_graph(n, 3)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b2a93101.py", line 22, in generate_d_regular_graph
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means it did not meet the pre-registered support condition and therefore cannot be supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14135
2	propose	ollama_remote	glm4:latest	0	0	12375
3	preregistration	ollama_remote	glm4:latest	0	0	9203
4	novelty	ollama_remote	glm4:latest	0	0	11097
5	novelty	ollama_remote	glm4:latest	0	0	9464
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26270
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13142
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13371
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11589
10	judge	ollama_remote	glm4:latest	0	0	12694

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133340 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/c266c5a3a6c4.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/c266c5a3a6c4.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/c266c5a3a6c4.tar.gz* (if generated)